Matemáticas para la ciencia de datos

Regresiones lineales simples

Carlos Yung
Versión inicial: Eo Jaehyun Kim

Tema 1

Temas a discutir: Eo Jaehyun Kim

- Conceptos básicos para ciencia de datos y aprendizaje automático
- Cómo obtener una línea de regresión a partir de un conjunto de datos
- Cómo se relaciona la covarianza con el modelo de regresión simple
- Cómo se relaciona la correlación con el R^2
- Cómo se relaciona todo lo anterior con la regresión lineal múltiple

En el curso, utilizamos varios lenguajes de programación y sistemas operativos para ofrecerle la posibilidad de trabajar con la herramienta que le resulte más cómoda. Por ejemplo, en este primer tema, utilizamos Python, Perl y Bash.

Definición del problema

El aprendizaje automático trata sobre el arte de investigación matemática de un algo que funciona. Por lo tanto, antes de hablar de cómo funciona algo, tenemos que ponernos de acuerdo en qué significa "funcionar". Para ello, tenemos que establecer un lenguaje para describir un problema específico.

Hay dos grupos de aprendizaje automático que también se conocen como:

- Aprendizaje supervisado: En el cuarto paradigma epistemológico (ejemplo por ejemplo).
- Aprendizaje no supervisado: Cuando no tenemos ejemplos de cómo funciona algo y tenemos que ir por el camino difícil.

Por lo tanto, en los ejemplos de aprendizaje supervisado, el objetivo, como en el paradigma de experimentos, que obtenemos información del entorno sobre la suposición de la etiqueta o corrente de salida de un determinado ejemplo.

Por lo tanto, en los ejemplos de aprendizaje supervisado, partimos de un conjunto de datos de entrenamiento, entrenamos, y el aprendizaje por refuerzo se fundamenta justo en el paradigma de la ciencia experimental. Por lo tanto, en los ejemplos de aprendizaje supervisado, el objetivo es que nuestro modelo sea capaz de generalizar, es decir, que nuestro modelo funcione bien con ejemplos que no son exactamente los mismos que nuestro conjunto de datos de entrenamiento.

Versiones y matrices con Numpy

Tomo los datos de experimentos de Kaggle de ejemplo. Aquí mostramos algunas formas de cambiar matrices y vectores con `numpy`:

```
[yung@fedora ~]$ cat -n *.py
     1  import numpy as np
     2
     3  tamano = np.shape(y)
     4  print("Tamano: ", tamano)
     5
     6  print("Y[0]: ", y[0])
     7
     8  print("Y[:5]: ", y[:5])
     9
    10  print("Y[:,0]: ", y[:,0])
    11
    12  print("Y[:,1]: ", y[:,1])
    13
    14  print("Y[:,:2]: ", y[:,:2])
    15
    16  print("Y[:5,:2]: ", y[:5,:2])
    17
    18  print("Y[:5,1]: ", y[:5,1])
    19
    20  print("Y[:5,1].reshape(5,1): ", y[:5,1].reshape(5,1))
    21
    22  print("Y[:5,1].reshape(1,5): ", y[:5,1].reshape(1,5))
[yung@fedora ~]$
[yung@fedora ~]$ python -W ignore *.py
Tamano:  (2000, 2)
Y[0]:  [0.0000000 1.0000000]
Y[:5]:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]]
Y[:,0]:  [0.0000000 0.0000000 0.0000000 ... 0.9999000 0.9999000 0.9999000]
Y[:,1]:  [1.0000000 1.0000000 1.0000000 ... 0.0099990 0.0099990 0.0099990]
Y[:,:2]:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 ...
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]]
Y[:5,:2]:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]]
Y[:5,1]:  [1.0000000 1.0000000 1.0000000 1.0000000 1.0000000]
Y[:5,1].reshape(5,1):  [[1.0000000]
 [1.0000000]
 [1.0000000]
 [1.0000000]
 [1.0000000]]
Y[:5,1].reshape(1,5):  [[1.0000000 1.0000000 1.0000000 1.0000000 1.0000000]]
[yung@fedora ~]$
[yung@fedora ~]$ cat -n *.py
     1  import numpy as np
     2
     3  tamano = np.shape(y)
     4  print("Tamano: ", tamano)
     5
     6  print("Y[0]: ", y[0])
     7
     8  print("Y[:5]: ", y[:5])
     9
    10  print("Y[:,0]: ", y[:,0])
    11
    12  print("Y[:,1]: ", y[:,1])
    13
    14  print("Y[:,:2]: ", y[:,:2])
    15
    16  print("Y[:5,:2]: ", y[:5,:2])
    17
    18  print("Y[:5,1]: ", y[:5,1])
    19
    20  print("Y[:5,1].reshape(5,1): ", y[:5,1].reshape(5,1))
    21
    22  print("Y[:5,1].reshape(1,5): ", y[:5,1].reshape(1,5))
[yung@fedora ~]$
[yung@fedora ~]$ python -W ignore *.py
Tamano:  (2000, 2)
Y[0]:  [0.0000000 1.0000000]
Y[:5]:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]]
Y[:,0]:  [0.0000000 0.0000000 0.0000000 ... 0.9999000 0.9999000 0.9999000]
Y[:,1]:  [1.0000000 1.0000000 1.0000000 ... 0.0099990 0.0099990 0.0099990]
Y[:,:2]:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 ...
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]]
Y[:5,:2]:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]]
Y[:5,1]:  [1.0000000 1.0000000 1.0000000 1.0000000 1.0000000]
Y[:5,1].reshape(5,1):  [[1.0000000]
 [1.0000000]
 [1.0000000]
 [1.0000000]
 [1.0000000]]
Y[:5,1].reshape(1,5):  [[1.0000000 1.0000000 1.0000000 1.0000000 1.0000000]]
[yung@fedora ~]$
```

La familia de modelos lineales

Comenzamos a hablarles sobre la familia lineal de regresiones lineales (mal generalizadas GLM). Mediante transformaciones, matrices y métodos de optimización, los MLG pueden resolver una amplia gama de problemas matemáticos.

Hoy, en clase, no nos preocuparemos tanto de las matrices como de otras cosas matemáticas. Es decir, nos preocuparemos un poco de todas de la gama familiar de los MLG y, sobre todo, de la regresión lineal. Es ingente que, en el mundillo de la inteligencia artificial, se hable tanto de redes neuronales y de aprendizaje profundo, cuando la mayoría de problemas se resuelven con regresiones lineales.

Preparación para descubrir una regresión lineal simple

Vamos a tomar como ejemplo unos datos de unos experimentos de Kaggle. Nuestro objetivo es encontrar una función que describa la relación entre los datos de entrada y los de salida. Para ello, tenemos que:

- Cada observación es una serie de números (el estímulo predictor) en el grupo de entrada, y cada observación es un número en el grupo de salida (la respuesta a ese estímulo predictor).
- Queremos aplicarles a un conjunto de datos de entrenamiento un operador lineal y el significado lineal del valor resultante de la ecuación en el momento del uso).
- Queremos que, en el momento del uso, nuestro modelo funcione bien.

Para ello, tenemos que:

- Normalizamos debido a estímulos (una serie de estímulos predictores) en un grupo de entrada.
- Normalizamos debido a respuestas (una serie de respuestas) en un grupo de salida.
- Tomamos el operador lineal.
- Comprobamos que funciona bien.

```
[yung@fedora ~]$ cat -n *.py
     1  import numpy as np
     2
     3  print("Cargando los ficheros de entrenamiento")
     4  y = np.loadtxt("Y.txt")
     5  x = np.loadtxt("X.txt")
     6
     7  print("Normalizando los datos")
     8  mx = x.mean(axis=0)
     9  sx = x.std(axis=0)
    10  x = (x - mx) / sx
    11  my = y.mean(axis=0)
    12  sy = y.std(axis=0)
    13  y = (y - my) / sy
    14
    15  print("Tomando el operador lineal")
    16  ones = np.ones(x.shape[0],dtype=x.dtype)
    17  print("ones: ", ones)
    18  print("ones.shape: ", ones.shape)
    19  print("x.shape: ", x.shape)
    20  print("x[:,0].shape: ", x[:,0].shape)
    21  print("x[:,0].reshape(x.shape[0],1).shape: ", x[:,0].reshape(x.shape[0],1).shape)
    22  print("x[:,0].reshape(x.shape[0],1): ", x[:,0].reshape(x.shape[0],1))
    23  print("ones.reshape(x.shape[0],1): ", ones.reshape(x.shape[0],1))
    24  print("x[:,1].reshape(x.shape[0],1): ", x[:,1].reshape(x.shape[0],1))
    25  x = np.concatenate(ones.reshape(x.shape[0],1),x[:,0].reshape(x.shape[0],1),x[:,1].reshape(x.shape[0],1),axis=1)
    26  print("Comprobando que funciona bien")
    27  print("x: ", x)
    28  print("x.shape: ", x.shape)
[yung@fedora ~]$
[yung@fedora ~]$ python -W ignore *.py
Cargando los ficheros de entrenamiento
Normalizando los datos
Tomando el operador lineal
ones:  [1.0000000 1.0000000 1.0000000 ... 1.0000000 1.0000000 1.0000000]
ones.shape:  (2000,)
x.shape:  (2000, 2)
x[:,0].shape:  (2000,)
x[:,0].reshape(x.shape[0],1).shape:  (2000, 1)
x[:,0].reshape(x.shape[0],1):  [[0.0000000]
 [0.0000000]
 [0.0000000]
 ...
 [0.9999000]
 [0.9999000]
 [0.9999000]]
ones.reshape(x.shape[0],1):  [[1.0000000]
 [1.0000000]
 [1.0000000]
 ...
 [1.0000000]
 [1.0000000]
 [1.0000000]]
x[:,1].reshape(x.shape[0],1):  [[1.0000000]
 [1.0000000]
 [1.0000000]
 ...
 [0.0099990]
 [0.0099990]
 [0.0099990]]
Comprobando que funciona bien
x:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 ...
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]]
x.shape:  (2000, 2)
[yung@fedora ~]$
```

Como podemos observar la dimensión tiene sentido positivo y balanceado. Para más detalles de una dimensión, podemos ejecutar `numpy.ndim`:

```
[yung@fedora ~]$ cat -n *.py
     1  import numpy as np
     2
     3  print("Cargando los ficheros de entrenamiento")
     4  y = np.loadtxt("Y.txt")
     5  x = np.loadtxt("X.txt")
     6
     7  print("Normalizando los datos")
     8  mx = x.mean(axis=0)
     9  sx = x.std(axis=0)
    10  x = (x - mx) / sx
    11  my = y.mean(axis=0)
    12  sy = y.std(axis=0)
    13  y = (y - my) / sy
    14
    15  print("Tomando el operador lineal")
    16  ones = np.ones(x.shape[0],dtype=x.dtype)
    17  print("ones: ", ones)
    18  print("ones.shape: ", ones.shape)
    19  print("x.shape: ", x.shape)
    20  print("x[:,0].shape: ", x[:,0].shape)
    21  print("x[:,0].reshape(x.shape[0],1).shape: ", x[:,0].reshape(x.shape[0],1).shape)
    22  print("x[:,0].reshape(x.shape[0],1): ", x[:,0].reshape(x.shape[0],1))
    23  print("ones.reshape(x.shape[0],1): ", ones.reshape(x.shape[0],1))
    24  print("x[:,1].reshape(x.shape[0],1): ", x[:,1].reshape(x.shape[0],1))
    25  x = np.concatenate(ones.reshape(x.shape[0],1),x[:,0].reshape(x.shape[0],1),x[:,1].reshape(x.shape[0],1),axis=1)
    26  print("Comprobando que funciona bien")
    27  print("x: ", x)
    28  print("x.shape: ", x.shape)
[yung@fedora ~]$
[yung@fedora ~]$ python -W ignore *.py
Cargando los ficheros de entrenamiento
Normalizando los datos
Tomando el operador lineal
ones:  [1.0000000 1.0000000 1.0000000 ... 1.0000000 1.0000000 1.0000000]
ones.shape:  (2000,)
x.shape:  (2000, 2)
x[:,0].shape:  (2000,)
x[:,0].reshape(x.shape[0],1).shape:  (2000, 1)
x[:,0].reshape(x.shape[0],1):  [[0.0000000]
 [0.0000000]
 [0.0000000]
 ...
 [0.9999000]
 [0.9999000]
 [0.9999000]]
ones.reshape(x.shape[0],1):  [[1.0000000]
 [1.0000000]
 [1.0000000]
 ...
 [1.0000000]
 [1.0000000]
 [1.0000000]]
x[:,1].reshape(x.shape[0],1):  [[1.0000000]
 [1.0000000]
 [1.0000000]
 ...
 [0.0099990]
 [0.0099990]
 [0.0099990]]
Comprobando que funciona bien
x:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 ...
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]]
x.shape:  (2000, 2)
[yung@fedora ~]$
```

Como podemos observar la dimensión tiene sentido positivo y balanceado. Para más detalles de una dimensión, podemos ejecutar `numpy.ndim`:

```
[yung@fedora ~]$ cat -n *.py
     1  import numpy as np
     2
     3  print("Cargando los ficheros de entrenamiento")
     4  y = np.loadtxt("Y.txt")
     5  x = np.loadtxt("X.txt")
     6
     7  print("Normalizando los datos")
     8  mx = x.mean(axis=0)
     9  sx = x.std(axis=0)
    10  x = (x - mx) / sx
    11  my = y.mean(axis=0)
    12  sy = y.std(axis=0)
    13  y = (y - my) / sy
    14
    15  print("Tomando el operador lineal")
    16  ones = np.ones(x.shape[0],dtype=x.dtype)
    17  print("ones: ", ones)
    18  print("ones.shape: ", ones.shape)
    19  print("x.shape: ", x.shape)
    20  print("x[:,0].shape: ", x[:,0].shape)
    21  print("x[:,0].reshape(x.shape[0],1).shape: ", x[:,0].reshape(x.shape[0],1).shape)
    22  print("x[:,0].reshape(x.shape[0],1): ", x[:,0].reshape(x.shape[0],1))
    23  print("ones.reshape(x.shape[0],1): ", ones.reshape(x.shape[0],1))
    24  print("x[:,1].reshape(x.shape[0],1): ", x[:,1].reshape(x.shape[0],1))
    25  x = np.concatenate(ones.reshape(x.shape[0],1),x[:,0].reshape(x.shape[0],1),x[:,1].reshape(x.shape[0],1),axis=1)
    26  print("Comprobando que funciona bien")
    27  print("x: ", x)
    28  print("x.shape: ", x.shape)
[yung@fedora ~]$
[yung@fedora ~]$ python -W ignore *.py
Cargando los ficheros de entrenamiento
Normalizando los datos
Tomando el operador lineal
ones:  [1.0000000 1.0000000 1.0000000 ... 1.0000000 1.0000000 1.0000000]
ones.shape:  (2000,)
x.shape:  (2000, 2)
x[:,0].shape:  (2000,)
x[:,0].reshape(x.shape[0],1).shape:  (2000, 1)
x[:,0].reshape(x.shape[0],1):  [[0.0000000]
 [0.0000000]
 [0.0000000]
 ...
 [0.9999000]
 [0.9999000]
 [0.9999000]]
ones.reshape(x.shape[0],1):  [[1.0000000]
 [1.0000000]
 [1.0000000]
 ...
 [1.0000000]
 [1.0000000]
 [1.0000000]]
x[:,1].reshape(x.shape[0],1):  [[1.0000000]
 [1.0000000]
 [1.0000000]
 ...
 [0.0099990]
 [0.0099990]
 [0.0099990]]
Comprobando que funciona bien
x:  [[0.0000000 1.0000000]
 [0.0000000 1.0000000]
 [0.0000000 1.0000000]
 ...
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]
 [0.9999000 0.0099990]]
x.shape:  (2000, 2)
[yung@fedora ~]$
```

Como medir las relaciones lineales

Comenzamos hoy escribiendo un fichero de ejemplo con la extensión `dat`, como `ejemplo.dat`.

Tomamos estadísticamente las variables de la siguiente manera:

```
[yung@fedora ~]$ cat -n *.dat
     1  # Ejemplo de ejemplo para la estadística
     2
     3  # Variable independiente
     4  x = [1, 2, 3, 4, 5]
     5
     6  # Variable dependiente
     7  y = [2, 4, 6, 8, 10]
[yung@fedora ~]$
[yung@fedora ~]$ cat -n *.pl
     1  #!/usr/bin/perl
     2
     3  print("Ejecutando el ejemplo de ejemplo para la estadística\n");
     4  $fichero = "ejemplo.dat";
     5  $cmd = "cat -n $fichero | grep -v \"#\" | grep -v \"^[ *]*[0-9]*[ *]*[0-9]*[ *]*[0-9]*
```

El gráfico de dispersión resultante indica que hay muchos insectos alrededor de los problemas que representan exactamente, como una aproximación de que algo no va bien con la cantidad de agentes.

El primer lugar, hay algunos puntos que se encuentran fuera de la tendencia dominante que los otros forman. Estos son probablemente problemas que tienen muchos insectos, aproximadamente diez veces más PN. Por lo tanto, no es inusual, tener una distribución normal.

Ahora, la tarea de ajuste a una ecuación dispersa es, por supuesto, diferente cuando se trata de problemas que tienen muchos insectos. Por lo tanto, para ajustar el modelo, podemos eliminar los datos de los puntos anómalos y ejecutar el modelo de nuevo.

Antes de hacer esto, también podemos ver que los residuales, que son la diferencia entre la línea ajustada y los puntos representados, tienen una distribución de los datos por valores que no son iguales, pero que parecen ser la diferencia con una distribución que es normal.

Predicciones con un modelo de regresión

Después de ejecutar el modelo de regresión y de ajustar los datos, podemos ver cómo cambia el ajuste. Lo cuenta datos en este de cualquiera mostrado en la diferencia de los puntos con respecto a nuestro predictor, en nuestro caso, PN.

```
Call:
lm(formula = Insectos ~ Problemas, data = Data)

Residuals:
    Min      1Q  Median      3Q     Max
-1.5000 -0.5000  0.0000  0.5000  1.5000

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -0.5000     0.5000  -1.000    0.321
Problemas     0.9000     0.0500  18.000    0.000
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9487 on 8 degrees of freedom
Multiple R-squared,  0.9818,	Adjusted R-squared:  0.9795
F-statistic:  324 on 1 and 8 DF,  p-value: 1.11e-08
```

Se trata de un método de predicción y es simplemente la proporción de los predicciones es también R2 con otra cifra al cuadrado del 98% y de hecho.

```
> predictor_lm = predict(Insectos_lm, Data)
> Data_with_predictions = cbind(Data, predictor_lm)
> ggplot(Data_with_predictions, aes(x = Problemas, y = Insectos)) +
  geom_point() + geom_line(aes(y = predictor_lm))
```



```
> Data$.fitted = fitted(Insectos_lm)
> Data$.resid = resid(Insectos_lm)
> ggplot(Data, aes(x = .fitted, y = .resid)) +
  geom_point() + geom_hline(yintercept = 0)
```


Ahora, podemos ver que los residuales no siguen la misma apariencia que los otros gráficos de dispersión. Ahora, podemos ver que los residuales tienen una distribución normal.

Regresión con Scikit-learn

Como hemos visto al trabajar con el paquete Statsmodels, es posible construir un modelo lineal utilizando el paquete de aprendizaje automático más complejo como Scikit-learn.

```
> Insectos_lm = lm(Insectos ~ Problemas, data = Data)
> predictor_lm = predict(Insectos_lm, Data)
> Data_with_predictions = cbind(Data, predictor_lm)
> ggplot(Data_with_predictions, aligned = TRUE, aes(x = Problemas, y = Insectos)) +
  geom_point() + geom_line(aes(y = predictor_lm))
```



```
> Data$.fitted = fitted(Insectos_lm)
> Data$.resid = resid(Insectos_lm)
> ggplot(Data, aligned = TRUE, aes(x = .fitted, y = .resid)) +
  geom_point() + geom_hline(yintercept = 0)
```


Con Statsmodels hay convenientes del paquete Scikit-learn se que todos los modelos, sin importar su tipo de complejidad, empiezan los mismos métodos.

```
> Data$.fitted = fitted(Insectos_lm)
> Data$.resid = resid(Insectos_lm)
> ggplot(Data, aligned = TRUE, aes(x = .fitted, y = .resid)) +
  geom_point() + geom_hline(yintercept = 0)
```



```
> Data$.fitted = fitted(Insectos_lm)
> Data$.resid = resid(Insectos_lm)
> ggplot(Data, aligned = TRUE, aes(x = .fitted, y = .resid)) +
  geom_point() + geom_hline(yintercept = 0)
```


Con el método de ajuste de predicción y especificaciones los 1 0 y 0 para entrenar con los datos. Y, por supuesto, podemos ver que los residuales siguen la misma apariencia que los otros gráficos de dispersión.

Resumen

En este capítulo hemos presentado la regresión lineal como alguna de aprendizaje automático.

Explicamos la forma Scikit-learn, a diferencia de los modelos estadísticos de Statsmodels, es conducido en mediante constructor de modelos de regresión lineal simple como la clase de la función de la función de regresión.

Después de hacer esto, Explicamos la diferencia en ellas a como funciona la regresión lineal en el aprendizaje de máquinas frente a los métodos de regresión lineal simple como la clase de la función de la función de regresión.

¡Ahora que ya lo entendiste!